



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto: Sistema Verificador de Cumplimiento
OWASP**

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

- Andia Navarro, Diego Fabrizio (2022073906)
- Concha Llaca, Gerardo Alejandro (2017057849)

Tacna – Perú

2026

Sistema Verificador de Cumplimiento OWASP

Informe de Factibilidad

Versión 1.0

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Andia Navarro, D. / Concha Llaca, G.	Mag. Ing. Patrick Cuadros Quiroga	Mag. Ing. Patrick Cuadros Quiroga	04/07/2026	Versión Original

ÍNDICE

1. Descripción del Proyecto	4
1.1 Nombre del proyecto	4
1.2 Duración del proyecto	4
1.3 Descripción	4
1.4 Objetivos	4
1.4.1 Objetivo general	4
1.4.2 Objetivos específicos	4
2. Riesgos	5
3. Análisis de la Situación actual	6
3.1 Planteamiento del problema	6
3.2 Consideraciones de hardware y software	6
4. Estudio de Factibilidad	7
4.1 Factibilidad Técnica	7
4.2 Factibilidad Económica	7
4.2.1 Costos Generales	7
4.2.2 Costos operativos durante el desarrollo	8
4.2.3 Costos del ambiente	8
4.2.4 Costos de personal	8
4.2.5 Costos totales del desarrollo del sistema	8
4.3 Factibilidad Operativa	9
4.4 Factibilidad Legal	9
4.5 Factibilidad Social	9
4.6 Factibilidad Ambiental	9
5. Análisis Financiero	10
5.1 Justificación de la Inversión	10
5.1.1 Beneficios del Proyecto	10
5.1.2 Criterios de Inversión	10
5.1.2.1 Relación Beneficio/Costo (B/C)	10
5.1.2.2 Valor Actual Neto (VAN)	10
5.1.2.3 Tasa Interna de Retorno (TIR)	10
6. Conclusiones	11

1. Descripción del Proyecto

1.1 Nombre del proyecto

Sistema Verificador de Cumplimiento OWASP (OWASP Verificator).

1.2 Duración del proyecto

El desarrollo del proyecto se ha planificado para una duración total de 12 semanas (3 meses), distribuidas en las fases de análisis, diseño, construcción, pruebas y despliegue, en el marco del ciclo académico 2026-I del curso de Calidad y Pruebas de Software.

1.3 Descripción

El proyecto consiste en el diseño e implementación de un ecosistema verificador de cumplimiento OWASP conformado por dos componentes complementarios: (1) una extensión para Visual Studio Code que realiza análisis estático de código en tiempo real y muestra un dashboard interactivo de cumplimiento, y (2) un sistema web con API RESTful (FastAPI) que permite auditar código, URLs de producción y repositorios de GitHub, verificando la presencia de vulnerabilidades del OWASP Top 10 y de dependencias con CVEs conocidos.

El proyecto surge para atender la necesidad de Securify Software Solutions S.A.C., una consultora de ciberseguridad y desarrollo seguro de software, de contar con una herramienta propia, ligera y de bajo costo que permita a sus equipos de desarrollo y auditoría detectar vulnerabilidades de seguridad de forma temprana, sin depender de soluciones SAST comerciales costosas como Checkmarx o Veracode. El sistema se desenvuelve en un contexto de creciente adopción de prácticas DevSecOps, en el que se busca integrar la seguridad desde las fases más tempranas del ciclo de vida del software.

1.4 Objetivos

1.4.1 Objetivo general

Diseñar e implementar un sistema verificador que permita evaluar de manera ágil y en tiempo real el cumplimiento de los controles de seguridad OWASP Top 10 en código fuente, repositorios y URLs de producción, integrando interfaces web, una API REST y una extensión para el entorno de desarrollo (IDE) Visual Studio Code.

1.4.2 Objetivos específicos

- Diseñar una plataforma accesible (dashboard web y extensión de IDE) que permita a los desarrolladores realizar análisis estáticos de código de forma rápida e intuitiva, logrando reducir el tiempo de detección de vulnerabilidades durante la codificación.
- Incorporar un motor de análisis estático basado en expresiones regulares y en el cotejo de dependencias contra una base de datos de CVEs, logrando alertar al desarrollador sobre fallos críticos del OWASP Top 10 antes de subir el código a producción.

- Proveer a los administradores y auditores una herramienta centralizada para gestionar usuarios, emitir y revocar tokens de API, y monitorear el historial de escaneos, logrando centralizar la trazabilidad de las auditorías de seguridad.
- Garantizar la portabilidad del sistema en entornos Windows, Linux y macOS mediante una persistencia dual SQLite3/MySQL, logrando que el sistema no dependa de infraestructuras de base de datos complejas.
- Implementar un archivo de firmas de reglas modular y extensible, logrando que los equipos de desarrollo personalicen los patrones de análisis según sus propias políticas internas de seguridad.

2. Riesgos

A continuación, se señalan los principales riesgos que pudieran afectar el éxito del proyecto:

- Falsos positivos o falsos negativos en el motor de análisis basado en expresiones regulares, que podrían restar confiabilidad a los reportes generados.
- Cambios en las APIs de terceros utilizadas (GitHub API, paneles de chat de GitHub Copilot o Google Gemini), que podrían romper la integración de remediación asistida por IA.
- Indisponibilidad o degradación del servicio en la infraestructura en la nube (Azure App Service / máquina virtual Debian) que aloja la API y el dashboard web.
- Equipo de desarrollo reducido (dos integrantes), lo que limita la capacidad de paralelizar tareas y podría generar cuellos de botella frente a los plazos académicos establecidos.
- Desactualización de la base de datos de CVEs empleada para el escaneo de dependencias, lo que podría dejar sin detectar vulnerabilidades recientes.
- Riesgos de seguridad propios del sistema (por ejemplo, uso indebido de tokens de API), que exigen controles de autenticación robustos desde el diseño.

3. Análisis de la Situación actual

3.1 Planteamiento del problema

Los desarrolladores de software modernos enfrentan constantes ciberamenazas que vulneran la privacidad y estabilidad de los sistemas que construyen. A pesar de la existencia de marcos de referencia como el OWASP Top 10, la falta de retroalimentación inmediata durante la codificación provoca que se introduzcan vulnerabilidades básicas de manera involuntaria, tales como la exposición de secretos y credenciales en texto plano, el uso de funciones inseguras susceptibles a inyección (eval, exec), la incorporación de dependencias desactualizadas con CVEs públicos, y el despliegue de endpoints web sin cabeceras HTTP de protección (CSP, HSTS, X-Frame-Options).

Las herramientas SAST comerciales existentes en el mercado (Checkmarx, Veracode, entre otras) resultan excesivamente costosas y complejas de configurar para equipos pequeños, consultoras medianas o desarrolladores independientes, lo que constituye la necesidad concreta que el proyecto busca resolver: ofrecer una alternativa ágil, económica e integrada al flujo de trabajo diario del programador.

3.2 Consideraciones de hardware y software

Para la implementación del sistema se evaluó la tecnología disponible y alcanzable por el equipo de desarrollo y por la organización piloto (Securify Software Solutions S.A.C.), terminándose viable el uso de las siguientes herramientas:

- Hardware: estaciones de trabajo de desarrollo estándar (procesador Intel/AMD de 4 núcleos, 8 GB de RAM como mínimo) y una máquina virtual en la nube (Azure App Service / VM Debian) para el hospedaje del backend y la base de datos en producción.
- Software backend: Python 3.11+, framework FastAPI, motor de plantillas Jinja2 y CSS puro para la interfaz web.
- Persistencia: SQLite3 para entornos de desarrollo local y MySQL para el entorno de producción.
- Extensión de IDE: Visual Studio Code (Extension API), publicada en Visual Studio Code Marketplace y en Open VSX Registry.
- Control de versiones y CI/CD: Git, GitHub y GitHub Actions, con un flujo de despliegue automatizado hacia Azure Web App.
- Pruebas: Pytest para pruebas unitarias e integración; Behave para pruebas de comportamiento (BDD).
- Conectividad: acceso a internet de banda ancha, dominio propio y navegadores web modernos (Chrome, Edge, Firefox) para el acceso al dashboard.

4. Estudio de Factibilidad

El presente estudio de factibilidad tiene como resultado esperado determinar si el desarrollo del Sistema Verificador de Cumplimiento OWASP es viable desde el punto de vista técnico, económico, operativo, legal, social y ambiental. Para su elaboración se realizaron entrevistas con ingenieros de software, analistas de QA y arquitectos de seguridad de Securify Software Solutions S.A.C., además de la revisión de la documentación técnica de las tecnologías propuestas (FastAPI, VS Code Extension API, GitHub API). El presente estudio fue revisado y aprobado por el docente del curso, Mag. Ing. Patrick Cuadros Quiroga.

4.1 Factibilidad Técnica

Se realizó una evaluación de la tecnología actual existente y de su aplicabilidad al desarrollo e implantación del sistema. El equipo cuenta con experiencia previa en Python, FastAPI, JavaScript/TypeScript y en el desarrollo de extensiones para Visual Studio Code, lo que reduce la curva de aprendizaje del proyecto.

Hardware: no se requiere adquisición de servidores físicos; basta con una máquina virtual en la nube (Azure App Service, plan gratuito/básico para el piloto académico) y las estaciones de trabajo de los integrantes del equipo.

Software: Python 3.11+, FastAPI, Jinja2, Uvicorn, SQLite3/MySQL, Visual Studio Code, Node.js (para la extensión), Git/GitHub, GitHub Actions, y navegadores web modernos compatibles con HTML5/CSS3.

Dominio e infraestructura de red: el sistema se publica bajo un dominio/IP pública asignado por el proveedor de nube (<http://38.250.116.71:8000>), con acceso mediante protocolo HTTP/HTTPS y conexión a internet estable, sin requerir infraestructura de red física adicional.

En conclusión, la tecnología necesaria se encuentra disponible, es de bajo costo o gratuita en su mayoría (licencias open source) y el equipo posee la capacidad técnica para implementarla, por lo que el proyecto es técnicamente factible.

4.2 Factibilidad Económica

El propósito de este estudio es determinar los beneficios económicos del proyecto en contraposición con sus costos. Dado que la mayoría de las herramientas empleadas son de licencia gratuita o de uso académico, la inversión inicial en infraestructura informática es reducida.

4.2.1 Costos Generales

Concepto	Cantidad	Costo unitario (S/.)	Costo total (S/.)
Útiles de oficina (papel, impresiones, anillados)	1	60.00	60.00
Cartuchos de impresora	1	80.00	80.00
Internet (uso compartido, referencial)	3 meses	40.00	120.00

Costo total de costos generales: S/. 260.00

4.2.2 Costos operativos durante el desarrollo

Los costos operativos considerados corresponden al consumo de energía eléctrica y conectividad de las estaciones de trabajo utilizadas por el equipo durante las 12 semanas

de desarrollo, estimados en S/. 150.00 en conjunto, dado que no se requiere alquiler de oficina al desarrollarse el proyecto de forma remota.

4.2.3 Costos del ambiente

Concepto	Costo mensual (S/.)	Meses	Costo total (S/.)
Hospedaje en la nube (Azure App Service, plan básico)	0.00 (nivel gratuito académico)	3	0.00
Dominio / IP pública del servidor	0.00 (incluido por el proveedor)	3	0.00
Licencias de publicación en marketplaces (VS Code Marketplace / Open VSX)	0.00 (gratuitas)	-	0.00

Costo total del ambiente: S/. 0.00, al emplearse íntegramente herramientas y planes gratuitos o de nivel académico.

4.2.4 Costos de personal

El costo de personal se calcula en función de las horas de trabajo del equipo de desarrollo, conformado por dos integrantes que cumplen los roles de analista-desarrollador full-stack. Se considera una tarifa referencial de S/. 25.00 por

Rol	N.º de personas	Horas semanales	Semanas	Tarifa (S/./h)	Costo total (S/.)
Analista-Desarrollador Full-Stack	2	15	12	25.00	9,000.00

hora, con una dedicación de 15 horas semanales por integrante durante 12 semanas. **No se considera personal para la operación y funcionamiento posterior del sistema, únicamente para su desarrollo. Costo total de personal: S/. 9,000.00.**

4.2.5 Costos totales del desarrollo del sistema

Concepto	Costo (S/.)
Costos generales	260.00
Costos operativos	150.00
Costos del ambiente	0.00
Costos de personal	9,000.00
Costo total del proyecto	9,410.00

El pago del costo de personal se plantea de forma referencial como contraprestación única al finalizar el desarrollo, dado el carácter académico del proyecto; en un escenario real de contratación por Securify Software Solutions S.A.C., este monto sería asumido íntegramente por la organización.

4.3 Factibilidad Operativa

El sistema ofrece beneficios operativos claros: reduce el tiempo de detección de vulnerabilidades, centraliza la auditoría de seguridad y disminuye la dependencia de herramientas SAST comerciales costosas. Securify Software Solutions S.A.C., como organización piloto, cuenta con personal técnico capacitado (desarrolladores, analistas de QA y arquitectos de seguridad) capaz de operar y mantener el sistema en funcionamiento, dado que se apoya en tecnologías ampliamente documentadas (Python, FastAPI, VS Code).

Interesados (stakeholders) identificados:

- Desarrolladores independientes y equipos in-house.
- Líderes de TI y auditores de seguridad.
- Administradores del sistema (gestión de tokens y usuarios).
- Docentes y estudiantes de la Escuela Profesional de Ingeniería de Sistemas de la UPT.
- Pipelines y scripts de integración continua (clientes API / CI-CD).

4.4 Factibilidad Legal

El proyecto no presenta conflictos con restricciones legales vigentes. Se ha considerado el cumplimiento de la Ley N.º 29733, Ley de Protección de Datos Personales del Perú, y su reglamento, en cuanto al tratamiento de la información almacenada en los registros de escaneo. Asimismo, las tecnologías empleadas (FastAPI, Jinja2, extensión de VS Code) se distribuyen bajo licencias de código abierto compatibles con un uso académico y comercial (MIT/Apache 2.0), y el uso de la API de GitHub se sujeta a los términos de servicio de GitHub. No se identifican restricciones adicionales relacionadas con seguridad, conducta de negocio, empleo o adquisiciones que impidan la ejecución del proyecto.

4.5 Factibilidad Social

El proyecto promueve la cultura del desarrollo seguro (DevSecOps) entre desarrolladores, estudiantes y docentes, fomentando buenas prácticas de codificación segura y ética profesional en el manejo de información sensible (credenciales, tokens de acceso). No se identifican climas políticos, culturales o de conducta que representen un riesgo para la aceptación social del sistema; por el contrario, responde a una necesidad creciente en la comunidad de desarrollo de software.

4.6 Factibilidad Ambiental

El impacto ambiental del proyecto es mínimo. Al tratarse de un sistema alojado en infraestructura en la nube y de una extensión de software, no genera residuos físicos ni requiere consumo significativo de recursos naturales. Adicionalmente, la generación de reportes digitales (PDF/JSON) en reemplazo de auditorías impresas en papel contribuye a la reducción del uso de papel y, en consecuencia, a un menor impacto ambiental.

5. Análisis Financiero

El plan financiero se ocupa del análisis de ingresos y gastos asociados al proyecto desde la perspectiva de un despliegue piloto para Securify Software Solutions S.A.C., a fin de detectar oportunamente situaciones financieramente inadecuadas y estimar el resultado esperado del proyecto.

5.1 Justificación de la Inversión

5.1.1 Beneficios del Proyecto

Beneficios tangibles:

- Reducción de costos frente a licencias de herramientas SAST comerciales (Checkmarx, Veracode), estimadas en miles de dólares anuales.
- Disminución del tiempo dedicado a la corrección de vulnerabilidades detectadas tardíamente en producción.
- Ahorro en horas-hombre de auditoría manual de cabeceras HTTP y dependencias vulnerables.
- Beneficios intangibles:
- Mejora de la cultura de seguridad (DevSecOps) dentro de los equipos de desarrollo.
- Mayor confiabilidad y reputación frente a clientes de Securify Software Solutions S.A.C.
- Disponibilidad de información centralizada y trazable sobre el estado de cumplimiento OWASP del código.
- Valor agregado diferencial frente a competidores que no ofrecen integración directa con IA (Copilot/Gemini) para la remediación de vulnerabilidades.

5.1.2 Criterios de Inversión

5.1.2.1 Relación Beneficio/Costo (B/C)

Considerando un ahorro estimado anual de S/. 14,000.00 frente a licencias SAST comerciales y servicios de auditoría manual, y un costo total de desarrollo de S/. 9,410.00, la relación B/C resulta:

$$B/C = 14,000.00 / 9,410.00 \approx 1.49$$

Al ser la relación B/C mayor a uno, se acepta el proyecto como beneficioso desde el punto de vista costo-beneficio.

5.1.2.2 Valor Actual Neto (VAN)

Proyectando los flujos netos estimados a un año, con una tasa de descuento (COK) referencial del 12% anual, el VAN estimado del proyecto es positivo ($VAN > 0$), lo que indica que el proyecto genera valor económico adicional respecto a la inversión inicial y, por tanto, se acepta el proyecto.

5.1.2.3 Tasa Interna de Retorno (TIR)

La TIR estimada del proyecto se sitúa por encima del costo de oportunidad de capital (COK) considerado (12%), lo que confirma que la rentabilidad promedio anual generada por el capital invertido supera el costo de oportunidad, por lo que se acepta el proyecto desde el punto de vista de este criterio.

6. Conclusiones

- El estudio de factibilidad técnica confirma que el equipo cuenta con la experiencia y las herramientas tecnológicas necesarias (Python, FastAPI, VS Code Extension API) para desarrollar el sistema sin incurrir en inversiones significativas en infraestructura.
- El estudio de factibilidad económica evidencia un costo total de desarrollo de S/. 9,410.00, mayoritariamente concentrado en horas de personal, y una relación beneficio/costo favorable ($B/C \approx 1.49$).
- Los estudios de factibilidad operativa, legal, social y ambiental no revelan restricciones que impidan la implementación del proyecto.
- El análisis financiero (VAN positivo y TIR superior al costo de oportunidad) confirma la viabilidad económica y la sostenibilidad del proyecto en el mediano plazo.
- En consecuencia, se concluye que el proyecto Sistema Verificador de Cumplimiento OWASP es viable y factible en todas sus dimensiones evaluadas, por lo que se recomienda continuar con las siguientes etapas de especificación de requerimientos, diseño arquitectónico e implementación.